

บทที่ 3

เทคโนโลยีและเครื่องมือในการพัฒนา

3.1 เทคโนโลยีคอทเน็ต

คอทเน็ตเฟรมเวิร์ก คือโครงร่างการพัฒนาโปรแกรมคอมพิวเตอร์ ที่ถูกออกแบบมาเพื่อให้อำนวยความสะดวกในการพัฒนาโปรแกรมสมัยใหม่ที่ใช้งานในระบบเครือข่าย โดยมีเป้าหมายการทำงานหลัก 3 ข้อ ได้แก่

1. การพัฒนาโปรแกรมในรูปแบบของเว็บเซอร์วิส ซึ่งจะหลักของโปรแกรมต่างๆ ที่ใช้งานบนอินเทอร์เน็ต เว็บเซอร์วิสจะช่วยให้การติดต่อสื่อสาร ระหว่างแอปพลิเคชันบนอินเทอร์เน็ตนั้นง่ายขึ้น และเป็นระบบมากยิ่งขึ้น
2. เว็บเซอร์วิสขั้นพื้นฐานเช่น การตรวจสอบชื่อผู้ใช้ที่ล็อกอินเข้าสู่ระบบ มีการพัฒนาให้เป็นมาตรฐาน และสามารถนำไปใช้ได้ทั่วไปบนอินเทอร์เน็ต
3. เครื่องคอมพิวเตอร์และอุปกรณ์พกพาต่างๆ ที่ต่อเชื่อมกับอินเทอร์เน็ตได้ เช่น พีดีเอและโทรศัพท์มือถือ จะมีบทบาทและประโยชน์มากขึ้นเมื่อสามารถติดต่อใช้งานโปรแกรมต่างๆ บนอินเทอร์เน็ตได้

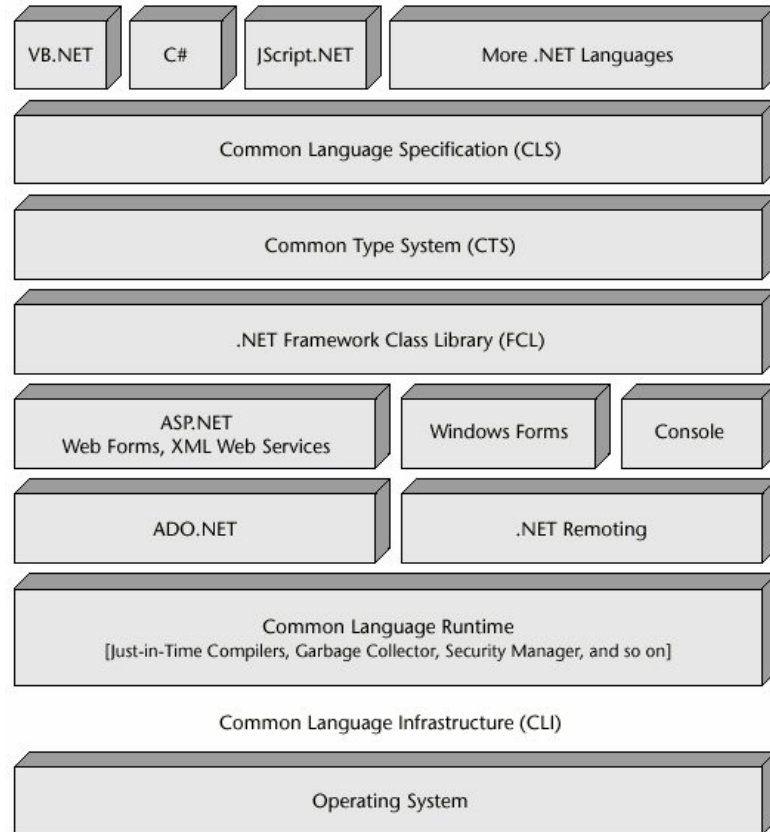
การที่จะทำให้ระบบหลายๆ ระบบทำงานต่อเชื่อมกันได้อย่างราบรื่นนั้น ไมโครซอฟท์ได้พัฒนารูปแบบการพัฒนาเว็บคอมพิวเตอร์ขึ้นมา เรียกว่า คอทเน็ตเฟรมเวิร์ก ซึ่งมีผู้คิดค้นพัฒนาจากหลายหน่วยงาน ยกตัวอย่างเช่น ชันไมโครซิสเต็มส์ บริษัทไอบีเอ็ม เป็นต้น แต่ว่า ไมโครซอฟท์ได้นำแนวคิดเหล่านั้นมาออกแบบให้อยู่ในรูปแบบที่สามารถต่อเชื่อมกันได้ง่ายขึ้น และเป็นระบบมากขึ้น โดยคอทเน็ตประกอบไปด้วยส่วนประกอบต่างๆ หลายส่วนด้วยกัน และส่วนประกอบเหล่านี้ถูกออกแบบมาเพื่อให้ทำงานได้เข้ากันได้ดียิ่งขึ้น

3.1.1 ส่วนประกอบของคอทเน็ตเฟรมเวิร์ก

ส่วนประกอบหลักของ คอทเน็ตเฟรมเวิร์กแบ่งเป็นชั้นๆ ได้แก่

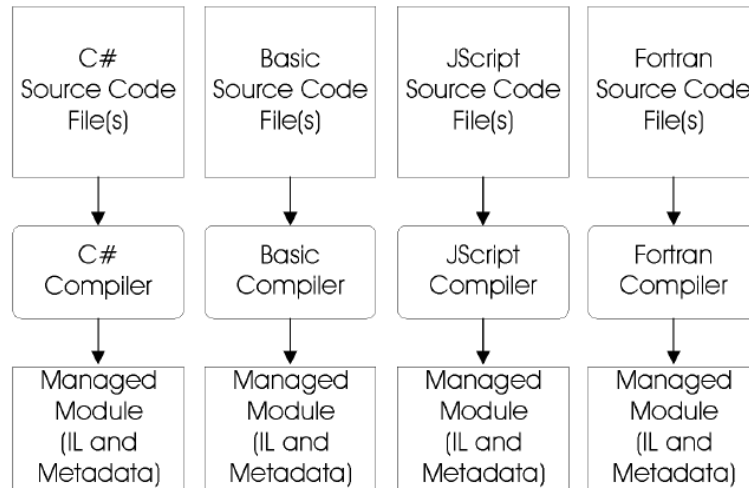
1. Common Language Runtime (CLR) ถือเป็นรากฐานของแพลตฟอร์มคอทเน็ต โดยเป็นส่วนพื้นฐานที่ติดต่อกับระบบปฏิบัติการวินโดวส์ ทำหน้าที่เป็น run-time environment ให้กับโปรแกรมที่เขียนขึ้นสำหรับใช้บนคอทเน็ต โดยเป็น execution engine ในการประมวลผลและจัดการโปรแกรมที่คอมไพล์แล้ว ให้ทำงานได้บนวินโดวส์ CLR มีส่วนของคอมไพเลอร์ ทั้งที่เป็นแบบปกติ คือคอมไพเลอร์ก่อนที่จะนำโปรแกรมไปใช้ และแบบ Just-In-Time คือคอมไพเลอร์เมื่อจะใช้โปรแกรมนั้นๆ หลังจากนั้นถ้าต้องการนำส่วนอื่นมาใช้งานอีกก็จะแปลงเพิ่มเฉพาะในส่วนนั้น ซึ่งช่วยให้โปรแกรมทำงานได้เร็วขึ้น เนื่องจากไม่ต้องรอให้แปลงเสร็จทั้งหมดก่อนจึงจะทำงานได้ มีส่วนของการจัดการหน่วยความจำ

ที่เอาไว้สำหรับจัดสรรหน่วยความจำของเครื่องให้กับโปรแกรม รวมไปถึงการจัดการสรรหน่วยความจำให้กับโปรแกรมต่างๆ และคืนหน่วยความจำที่ไม่ถูกใช้งานแล้วให้กับระบบ (Garbage Collection) การจัดการกับข้อผิดพลาดต่างๆ (Exception Handling) รวมถึงดูแลเรื่องความปลอดภัย (security management) ด้วย ส่วนของ Common Type Systems (CTS) ทำให้ภาษาต่าง ๆ ที่เขียนขึ้นบน .Net สามารถทำงานร่วมกันได้ เพราะขนาด และรูปแบบของข้อมูลที่เก็บไว้นั้นเป็นรูปแบบเดียวกัน



รูปที่ 3.1 แสดงสถาปัตยกรรมของคอทเน็ตเฟรมเวิร์ก

2. Base Classes เป็นคลาสไลบรารีพื้นฐาน ที่โปรแกรมต่างๆ ไม่ว่าจะเขียนด้วยภาษาใดบนคอทเน็ตก็สามารถใช้ร่วมกันได้ เช่น การติดต่อระบบฐานข้อมูล การติดต่อกับ ระบบไฟล์ของเซิร์ฟเวอร์ เป็นต้น
3. Programming Languages เป็นเซตของภาษาคอมพิวเตอร์ ที่ถูกออกแบบมาเพื่อการเขียนโปรแกรมบนคอทเน็ตเฟรมเวิร์ก โดยมีภาษาหลักๆ ได้แก่ VB.Net ซึ่งเป็นตัวที่พัฒนาต่อมาจาก VB C# ซึ่งเป็นภาษาใหม่ที่มีรูปแบบการเขียนใกล้เคียงกับ Java และ C++ Visual C++ และ JScript.Net ส่วนภาษาอื่นๆ นั้น มีบริษัท หรือหน่วยงานอื่นๆ เป็นผู้พัฒนาขึ้น ซึ่งมีอีกจำนวนหลายภาษา



รูปที่ 3.2 แสดงการแปลจากโค้ดให้อยู่ในรูปของ IL

สำหรับบนคอมพิวเตอร์เฟรมเวิร์กนั้นไม่ว่าจะเขียนโปรแกรมด้วยภาษาใดก็ตามคอมไพล์เลอร์ใน CLR ก็จะคอมไพล์โปรแกรมนั้นให้อยู่ในรูปของ Intermediate Language (IL) ซึ่งจะถูกนำไปแปลเป็นภาษาเครื่อง (Native Code) อีกทีเมื่อตอนที่นำไปใช้

3.2 หลักการของซ็อกเก็ต

โดยทั่วไปในการติดต่อสื่อสารที่เป็นที่นิยมใช้กันในปัจจุบัน จะกระทำบนพื้นฐานของ Protocol TCP/IP และชุดของ Software ซึ่งทำงานกับ Packets ตามมาตรฐานของ TCP/IP นั้นจะถูกสร้างขึ้นมาเป็นส่วนหนึ่งของตัวระบบปฏิบัติการ (Operating System, OS) ดังนั้น เมื่อ Application Software ใด ๆ ที่ต้องการสื่อสารผ่านระบบเครือข่ายตามมาตรฐาน TCP/IP Protocol จะต้องติดต่อกับระบบปฏิบัติการ เพื่อขอรับบริการ (Services)

สำหรับทางผู้พัฒนามาตรฐาน TCP/IP Protocol ไม่ต้องการให้ TCP/IP Protocol ใช้งานได้ดีกับ Platform ใดเป็นการเฉพาะ หรือใช้งานได้ดีกับเพียงระบบปฏิบัติการระบบใดระบบหนึ่ง ดังนั้นผู้พัฒนาจึงมีความระมัดระวังในการกำหนดมาตรฐาน มิให้มีการอ้างอิงข้อมูลการเชื่อมต่อภายในซึ่งเป็นรูปแบบเฉพาะของ Platform ใด Platform หนึ่งหรืออ้างอิงวิธีการเชื่อมต่อกับ Application Software ในลักษณะหรือรูปแบบเฉพาะของระบบปฏิบัติการจากผู้ผลิตรายใดรายหนึ่ง และด้วยแนวคิดในการออกแบบที่ไม่ยึดติดกับ Platform หรือระบบปฏิบัติการใด ๆ นี้ทำให้อาจกล่าวได้ว่า Protocol TCP/IP มีลักษณะเป็น Loosely Specified Protocol Software Interface

จากที่กล่าวมาข้างต้น สามารถกล่าวโดยสรุปได้ว่า “มาตรฐาน TCP/IP นั้นมิได้ระบุรายละเอียดเกี่ยวกับการเรียกใช้งาน TCP/IP Protocol Software ของ Application Software หากแต่มีเพียงข้อแนะนำเกี่ยวกับ functions ที่ควรจะมีในชุดของ TCP/IP Protocol Software เท่านั้น

โดยในส่วนของรายละเอียดที่ นอกเหนือจากนั้นจะเปิดกว้างให้นักออกแบบระบบเป็นกำหนด และลงรายละเอียดเอง”

ข้อดีของการออกแบบ TCP/IP ให้เป็นแบบ Loosely Specified Protocol Software Interface นั้นคือความ Flexible และ Tolerance ทั้งนี้เพราะการไม่กำหนดรายละเอียดปลีกย่อยทำให้นักออกแบบระบบสามารถที่จะใส่ TCP/IP ลงในระบบปฏิบัติการได้โดยง่ายไม่ว่าระบบปฏิบัติการนั้นจะเป็นระบบปฏิบัติการแบบง่าย ๆ อย่างที่ใช้กันใน Embedded System ไปจนถึงระบบปฏิบัติการที่สลับซับซ้อนอย่างในระบบปฏิบัติการที่ใช้ในเครื่อง Super Computer เป็นต้น นอกจากนี้ด้วยการที่ไม่กำหนดวิธีการเชื่อมต่อกับ Application Software ทำให้นักออกแบบระบบสามารถเลือกใช้รูปแบบในการเชื่อมต่อ (Interface) ได้อย่างอิสระ โดยสามารถใช้ได้ทั้งแบบ Procedural หรือ Message-Passing เป็นต้น

แต่ทั้งนี้ Loosely Specified Software Interface นั้นก็ยังมีข้อด้อยเช่นกัน กล่าวคือการที่นักออกแบบระบบมีอิสระในการใส่รายละเอียดในส่วนการเชื่อมต่อ (Interface) จึงทำให้เกิดความหลากหลายของรูปแบบการเชื่อมต่อ ซึ่งนั่นทำให้การเขียน Application Program เพื่อการใช้งาน TCP/IP มีความ ยุ่งยากและซับซ้อนมากยิ่งขึ้น อีกทั้งยังทำให้ Application Program นั้นยากแก่การ Port ไปยังระบบปฏิบัติการหรือ Platform อื่น ๆ

3.2.1 รูปแบบของ TCP/IP Software Interface

แม้ว่านักออกแบบระบบจะสามารถใส่ส่วนการเชื่อมต่อกับซอฟต์แวร์ (Software Interface) ใดๆก็ได้ แต่ในทางปฏิบัติกลับมีรูปแบบของ TCP/IP Software Interface อยู่เพียงไม่กี่แบบเท่านั้น และ TCP/IP Software Interface ที่นิยมใช้งานกันอย่างแพร่หลายจะมีอยู่เพียง 2 รูปแบบเท่านั้นคือ

1. Socket Interface เป็น TCP/IP Software Interface ที่พัฒนาขึ้นโดย University of California at Berkley ซึ่งในตอนต้นได้พัฒนารูปแบบการเชื่อมต่อแบบนี้เพื่อใช้กับระบบปฏิบัติการ Berkley Unix
2. Transport Layer Interface หรือ TLI เป็น TCP/IP Software Interface ที่พัฒนาโดย AT&T เพื่อใช้งานกับระบบปฏิบัติการ System V Unix

3.2.2 Interface Functionality

แม้ว่าทางผู้พัฒนา TCP/IP Protocol Software Interface จะไม่ได้มีการกำหนดรูปแบบหรือวิธีการใด ๆ ในการเชื่อมต่อ Application Software กับ TCP/IP Protocol Software แต่ทางผู้พัฒนาเองก็ได้มีข้อแนะนำเกี่ยวกับ Functions ต่าง ๆ ที่จำเป็น เป็นต้นว่า Interface จะต้องสามารถรองรับการทำงานต่าง ๆ ตามหลักการดังต่อไปนี้ เช่น การจองเนื้อที่ทรัพยากรเพื่อใช้ในการสื่อสารข้อมูล สามารถระบุตำแหน่ง end-point ของต้นทางและปลายทางได้ สามารถสถาปนาการ

เชื่อมต่อได้ในกรณีที่เป็นไคลเอนท์ สามารถรับการเชื่อมต่อได้ ในกรณีที่เป็นเซิร์ฟเวอร์ สามารถรับและส่งข้อมูลได้ บอกได้ว่าข้อมูลจะมาถึงเมื่อไหร่ มีความสามารถในการยุติการเชื่อมต่อ และจัดการได้เมื่อปลายทางต้องการยุติการเชื่อมต่อ เป็นต้น

นอกจาก Concept ในการทำงานดังที่ได้กล่าวมาแล้วข้างต้น ทางผู้พัฒนามาตรฐาน TCP/IP Protocol ได้สร้าง Conceptual Interface ขึ้นมาเพื่อเป็นแนวทางให้นักออกแบบระบบสามารถพัฒนาซอฟต์แวร์ของตนได้ง่ายและเร็วขึ้นโดย Conceptual Interface ที่ทางผู้พัฒนามาตรฐาน TCP/IP Protocol ได้สร้างขึ้นนั้นจะอยู่ในรูปของ Set of Procedures and Functions เหตุที่ Conceptual Interface อยู่ในรูปแบบของ Procedures และ Functions นั้น เนื่องจากในช่วงเวลาที่พัฒนา Conceptual Interface ระบบปฏิบัติการต่าง ๆ ในยุคนั้นนิยมใช้วิธีการซึ่งเรียกกันว่า Procedural Mechanism ในการเคลื่อนย้ายการควบคุม (Transfer control) จาก Application Software ไปยังระบบปฏิบัติการ ดังนั้นจึงอาจกล่าวโดยสรุปเกี่ยวกับ Conceptual Interface ได้ดังนี้ “Conceptual Interface ซึ่งถูกกำหนดโดยผู้พัฒนามาตรฐาน TCP/IP นั้นจะไม่ระบุรูปแบบของการแทนข้อมูล (Data representation) หรือรายละเอียดในการเขียนโปรแกรม หากแต่แสดงเพียงตัวอย่างหนึ่งของของการเชื่อมต่อ ซึ่งมีความเป็นไปได้ในทางปฏิบัติที่ระบบปฏิบัติการจะสามารถให้บริการแก่ Application Software ที่ต้องการใช้งาน TCP/IP Protocol”

3.2.3 Berkeley Sockets

ช่วงต้นทศวรรษที่ 1980 Advanced Research Projects Agency (ARPA) ได้ให้ทุนกับ University of California at Berkeley เพื่อพัฒนา TCP/IP Software บนระบบปฏิบัติการ UNIX และผลส่วนหนึ่งของโครงการดังกล่าวคือการออกแบบและพัฒนาส่วนการเชื่อมต่อ (Interface) ซึ่ง Application Software จะใช้ในการสื่อสารข้อมูลผ่านเครือข่าย และผลที่ได้รับคือ Berkeley Sockets Interface หรือที่เรียกกันสั้น ๆ ว่า Sockets Interface โดยจุดเด่นของ Sockets Interface นั้นคือการสนับสนุนการสื่อสารข้อมูลผ่านเครือข่ายด้วย Protocols ที่หลากหลายที่สุดเท่าที่จะเป็นไปได้ Sockets Interface ได้ถูกนำไปใส่ในระบบปฏิบัติการสำหรับเครื่อง Work station ระดับสูงจากผู้ผลิตรายใหญ่ อาทิเช่น Sun Microsystems Inc., Tektronix Inc., Digital Equipment Corp. เป็นต้น ทำให้ Sockets Interface กลายเป็น Interface ที่สำคัญ และมีผู้ใช้งานอย่างกว้างขวาง จนกระทั่งอาจกล่าวได้ว่า Sockets Interface นั้นได้กลายเป็นมาตรฐานสำหรับการเชื่อมต่อ (Interface) ไปโดยปริยาย

3.2.3.1 System Data Structure for Sockets

เมื่อมีการสร้าง Socket ใหม่ ระบบปฏิบัติการจะทำการ allocate พื้นที่ในหน่วยความจำส่วนหนึ่งเพื่อสร้าง Data Structure สำหรับเก็บข้อมูลซึ่งจำเป็นสำหรับการสื่อสารข้อมูลผ่านเครือข่าย ซึ่ง Data Structure นี้จะประกอบไปด้วย field หลาย field ด้วยกัน โดยปกติเมื่อ Sockets ถูกสร้างขึ้น fields ต่าง ๆ เหล่านี้ใน data structure นั้นจะถูกวางไว้ Application Program ที่สร้าง

Socket นั้นจะต้องเรียกใช้งาน function สำหรับใส่ข้อมูลต่าง ๆ ลงใน fields ที่เว้นว่างไว้เหล่านั้น เอง

3.2.3.2 Sockets Descriptors

โดยปกติในระบบปฏิบัติการจะมีการสร้าง Descriptor Table สำหรับเก็บ Data Structure ซึ่งเกี่ยวข้องกับ I/O ไว้ ซึ่งปกติ Descriptor Table สำหรับแต่ละ Process จะถูกเก็บแยกออกจากกัน โดยทั่วไป Descriptor Table จะมีลักษณะเป็น Array of Pointer ซึ่งชี้ไปยัง Data Structure ที่เกี่ยวข้องสำหรับในกรณีของ Sockets ก็จะเป็นเช่นเดียวกัน ทั้งนี้เพราะ Sockets นั้นนับเป็น I/O ประเภทหนึ่งด้วยเช่นกัน (I/O ซึ่งติดต่อกับเครือข่าย) ดังนั้นเมื่อมีการสร้าง Socket ก็จะมีการ update Descriptor Table โดยเพิ่ม Pointer ที่ชี้ไปยัง Data Structure ที่เก็บข้อมูลของ Sockets ที่สร้างขึ้นใหม่นั้น

3.2.3.3 การใช้งาน Sockets

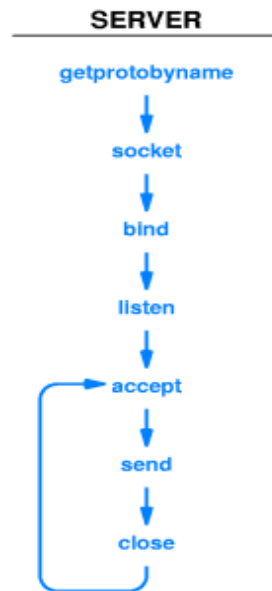
Sockets ที่ถูกสร้างขึ้นนั้นจะมีวัตถุประสงค์ในการใช้งานอยู่เพียง 2 ลักษณะได้แก่ การรอรับการเชื่อมต่อและการเชื่อมต่อไปยังเครื่องปลายทาง (Initiate a connection) โดย Sockets ที่รอรับการเชื่อมต่อนั้นจะเป็น Sockets ที่ใช้ใน Server Application ซึ่ง Sockets ชนิดนี้มีชื่อเรียกว่า Passive Socket ในขณะที่ Sockets ที่พยายามเชื่อมต่อไปยังเครื่องปลายทางซึ่งใช้ใน Client Application นั้นจะมีชื่อเรียกว่า Active Socket ทั้ง Active Sockets และ Passive Sockets นั้นในขั้นแรกจะถูกสร้างขึ้นมาด้วยวิธีการเดียวกัน ดังนั้นข้อแตกต่างของ Active Sockets และ Passive Sockets จึงอยู่ที่วัตถุประสงค์การใช้งานเท่านั้น

โดย Sockets สามารถแบ่งออกเป็นประเภทตามลักษณะการรับส่งข้อมูลผ่านเครือข่ายได้เป็น 3 ประเภท ดังนี้

1. Stream Sockets หรือ Connection Oriented Sockets
2. Datagram Sockets หรือ Connectionless Sockets
3. Raw Sockets

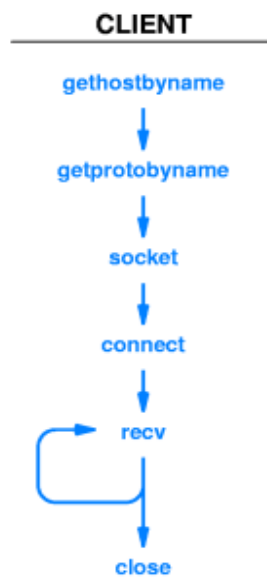
3.2.4 Working with Sockets

3.2.4.1 Passive Sockets หรือ Sockets ซึ่งใช้ใน Server Application นั้นมีขั้นตอนการสร้างดังนี้



รูปที่ 3.3 Diagram ขั้นตอนการสร้าง Passive Sockets

3.2.4.2 Active Sockets หรือ Socket ที่ใช้งานใน Client Application มีขั้นตอนการสร้างดังนี้



รูปที่ 3.4 Diagram ขั้นตอนการสร้าง Active Sockets

3.3 Java Applet

Applet เป็นโปรแกรมที่เขียนขึ้นโดยภาษา Java ซึ่งเวลาทำงานจะไม่เอ็กซีคิวต์เป็นโปรแกรมเดี่ยวๆ แต่จะต้องทำงานภายใต้โปรแกรมอื่นไม่สามารถควบคุมการดำเนินของตัวเองได้ทั้งหมด เช่น เว็บเบราว์เซอร์โดยการฝังไปกับเอกสาร HTML applet จะถูกเก็บไว้ที่เว็บเซิร์ฟเวอร์และจะถูกเรียกใช้งานโดยไคลเอนต์ผ่านทางโปรโตคอล HTTP เหมือนกับว่า applet เป็นส่วนหนึ่งในคำสั่ง HTML ของเว็บเพจนั้นเช่นเดียวกับการใส่รูปภาพเข้าไปในหน้าเว็บเพจ โดยจะใช้แท็ก <APPLET> และ </APPLET> เป็นตัวกำหนดขอบเขตของ applet ดังรูปที่ 3.8 ซึ่งคุณสมบัติการทำงานภายใต้เบราว์เซอร์นี้ทำให้สามารถสร้างโปรแกรมเพื่อให้ใครก็ได้เข้ามาใช้ผ่านทางหน้าเว็บเพจได้

```
<APPLET CODE="Appletclass.class" WIDTH="100" HEIGHT="50">
</APPLET>
```

รูปที่ 3.5 ตัวอย่างการใช้แท็ก <APPLET> ในเอกสาร HTML

เบราว์เซอร์ที่เอ็กซีคิวต์ applet ทั่วไปเรียกว่า applet container ตัวอย่างเช่น Appletviewer ซึ่งเป็น applet container ที่มาพร้อมกับ Java 2 Software Development Kit (J2SDK) มีไว้สำหรับใช้ทดสอบ applet ที่กำลังอยู่ในขั้นตอนการพัฒนา ก่อนที่จะนำไปใส่ไว้ในเว็บเพจ ในขณะที่เราใช้เว็บเบราว์เซอร์ที่อนุญาตให้ใช้เทคโนโลยี Java ในการแสดงหน้าเว็บที่มี applet ฝังมาด้วย โค้ด applet จะถูกส่งมายังระบบของเราและจะถูกเอ็กซีคิวต์โดย Java Virtual Machine (JVM) ของเว็บเบราว์เซอร์

3.3.1 ข้อจำกัดของ applet

เนื่องจาก applet เป็นโปรแกรมที่ถูกแจกจ่ายไปในระบบเครือข่าย ดังนั้นจึงต้องมีข้อจำกัดบางอย่างเพื่อไม่ให้เกิดการนำเอา applet ไปใช้ในทางที่จะก่อให้เกิดความเสียหายกับข้อมูลหรือระบบของเครื่องคอมพิวเตอร์ที่รับ applet นั้นไปทำงาน

โดยปกติแล้วแล้วเบราว์เซอร์จะถือว่า applet ที่รับมาจากเครื่องอื่นในระบบเครือข่ายเป็นโปรแกรมที่น่าไว้วางใจ (untrusted applet) และจะให้ applet ทำงานภายใต้สภาพแวดล้อมที่ปลอดภัยซึ่งมีการติดตั้ง Security Manager ให้คอยตรวจสอบการทำงานของ applet และไม่ยอมให้มีการละเมิดข้อจำกัด

ข้อจำกัดของ applet ถ้ามีการนำเอาไปใช้ จะเป็นตัวที่ทำให้ applet ไม่น่าไว้วางใจ ซึ่งเว็บเบราว์เซอร์แต่ละตัวอาจจะมีข้อจำกัดที่แตกต่างกันไป และอาจจะยอมให้ผู้ผู้ใช้เลือกผ่อนปรนข้อจำกัดบางอย่างได้ แต่อย่างไรก็ตามเราก็ต้องถือว่า untrusted applet ถูกจำกัดไม่ให้มีกระทำดังต่อไปนี้ [5]

1. ข้อจำกัดเกี่ยวกับระบบไฟล์ของเครื่องที่รับ applet นั้นไปทำงาน (local filesystem)
 - ห้ามเขียน อ่าน หรือลบไฟล์ หรือแม้แต่แก้ไข permission ของไฟล์
 - ห้ามสร้างไดเรกทอรีหรือการจัดการเกี่ยวกับไดเรกทอรี
 - ห้ามเขียนหรืออ่านไฟล์จากออปเจ็ก FileDescriptor
2. ข้อจำกัดเกี่ยวกับระบบเครือข่าย
 - ห้ามสร้างการเชื่อมต่อไปยังเครื่องอื่นที่ไม่ใช่เครื่องที่ applet นั้นถูกโหลดมา
 - ห้าม listen หรือ accept ในพอร์ต (port) ที่หมายเลขน้อยกว่าหรือเท่ากับ 1024
 - ห้ามให้ multicast socket
 - สร้างหรือรีจิสเตอร์ SocketImplFactory, URLStreamHandlerFactory หรือ ContentHandlerFactory
3. ไม่สามารถใช้ฟังก์ชันบางอย่างของระบบได้
 - ออกจาก Java interpreter โดยการเรียกใช้คำสั่ง System.exit() หรือคำสั่ง Runtime.exit()
 - สร้างโปรเซสใหม่โดยการเรียกผ่านทางคำสั่ง Runtime.exec()
 - โหลด native code library ด้วยการเรียกให้เมทอด load() หรือ loadLibrary() ของคลาส Runtime หรือคลาส System

นอกจากนี้ยังมีข้อจำกัดของ applet อีกมากแต่จะขอยกมาให้เห็นเพียงเท่านี้ก่อน

เว็บเบราว์เซอร์และ applet viewer อาจจะมีการผ่อนปรนข้อจำกัดบางอย่างให้กับ applet ที่ถูกโหลดมาจาก local filesystem บ้าง เพราะถือว่า local applet ต้องมีความน่าไว้วางใจมากกว่า applet ที่ถูกโหลดมาจากเครือข่ายและให้ถือเป็นความรับผิดชอบของผู้ใช้เอง

3.3.2 Signed Applet

ใน Java 1.1 ได้มีการเพิ่มความสามารถในการลงลายมือชื่อดิจิทัล (digital signature) ให้กับไฟล์ .jar (Java archive file) ซึ่งเป็นไฟล์ที่เก็บรวมไฟล์ .class และไฟล์ที่จำเป็นอื่นๆ ของ applet ไว้ในไฟล์เพียงไฟล์เดียวด้วย ลายมือชื่อดิจิทัลนี้ใช้ระบุถึงเจ้าของหรือแหล่งที่มาของ applet ซึ่งเราสามารถใช้การตัดสินใจได้ว่า applet นั้นมีความน่าไว้วางใจมากน้อยแค่ไหน ถ้าเราไว้วางใจในแหล่งที่มาของ applet เราก็สามารถที่จะบอกให้เว็บเบราว์เซอร์หรือ applet viewer ว่า applet นั้นมีความน่าไว้วางใจ (trusted applet) ซึ่งจะทำให้ไม่ถูกจำกัดเหมือน untrusted applet

กระบวนการของลงลายมือชื่อดิจิทัลลงไปในไฟล์ .jar ของ applet ขึ้นอยู่กับแพลตฟอร์ม โดยใน Java 1.1 จะใช้โปรแกรม javakey ในขณะที่ Java 1.2 ใช้ jarsigner แทน Netscape และ Microsoft ต่างก็มีโปรแกรมที่ใช้สำหรับลงลายมือชื่อดิจิทัลเองเหมือนกัน เพื่อให้เหมาะสมกับเบราว์เซอร์ของตนเอง

3.4 Javascript

ในการสร้างเว็บเพจที่ต้องการให้มีการโต้ตอบกับผู้ใช้ได้อย่างทันทีทันใดนั้น สามารถทำได้โดยใช้ภาษาสคริปต์ (Scripting Language) ร่วมกับการทำงานของ HTML ภาษาสคริปต์นี้เป็นชุดคำสั่งที่ไม่ใช่ HTML แต่ถูกแทรกไว้ในส่วนหนึ่งเอกสาร HTML และสามารถทำงานร่วมกับคำสั่งหรือแท็กของ HTML ได้

Javascript ก็เป็นภาษาสคริปต์ภาษาหนึ่งที่พัฒนาขึ้นโดย Sun และแต่เดิมเรียกว่า Livescript และได้ถูกเปลี่ยนชื่อเป็น Javascript เมื่อปี ค.ศ. 1995 โครงสร้างภาษาของ Javascript มีความคล้ายคลึงกับ Java มาก โดย Javascript เป็น complement ของ Java สามารถติดต่อกับส่วนต่างๆ ของ Java Applet โดยสคริปต์ที่เขียนขึ้นมาได้ คำสั่งของ Javascript สามารถนำมาใช้แสดงกำหนดคุณสมบัติ สอบถามสถานะ หรือควบคุมการทำงานของ Applet และปลั๊กอิน นอกจากนี้ Javascript ยังสนับสนุนรูปแบบนิพจน์และการควบคุมพื้นฐานของภาษา Java อีกด้วย

Javascript ได้ถูกออกแบบมาเพื่อใช้เป็นส่วนเพิ่มขยายในภาษา HTML โดยเฉพาะ ช่วยให้สามารถควบคุมเว็บเพจได้อย่างง่ายดายเหมาะกับการทำงานอย่างรวดเร็ว และเน้นที่ความถูกต้องเป็นสำคัญ เนื่องจาก Javascript เป็นภาษาแบบ interpreter ดังนั้นเราจึงสามารถใช้งาน Javascript ร่วมกับเอกสาร HTML โดยแทรกสคริปต์ลงไปในหน้าเอกสาร HTML ได้เลย โดยจะต้องระบุส่วนที่เป็น Javascript ด้วยแท็ก <SCRIPT LANGUAGE="Javascript"> ดังรูปที่ เลขก็ได้ ซึ่งในขณะที่เบราว์เซอร์ทำงานจะส่งสคริปต์นี้ไปให้ตัวแปลสคริปต์อีกทีหนึ่ง

```
<script language="javascript">
<!--
function popup(mylink, windowname)
{
  if (! window.focus) return true;
  var href;
  if (typeof(mylink) == 'string')
    href=mylink;
  else
    href=mylink.href;
  window.open(href, windowname, 'width=950, height=550, scrollbars=no');
  return false;
}
-->
</script>
```

รูปที่ 3.6 ตัวอย่าง Javascript สำหรับใช้แสดงหน้าเว็บเป็นแบบ popup window